TOOLKIT · BACKEND ENGINEERING HANDBOOK

JUNIT 5 COMPLETE HANDBOOK

Unit Testing, Spring Boot Slices, Mockito, Integration Testing & Production Mastery

Phase	Tool ID	Curriculum Scope	Deliverable Type
I — Developer Foundations	**Tool 05 of 13 (JUnit 5)**	**2 Comprehensive Chunks**	**Publication Handbook**
Interview Questions	**Practical Labs**	**Engineering Framework**	**Target Audience**
30 Rigorous Questions	**Production Testing Labs**	**3-Layer Architecture**	**Backend Engineers & SDE-1**

Executive Notice: This handbook is the definitive automated Java testing volume from the *ToolKit* curriculum. It provides comprehensive coverage of JUnit 5 (Platform, Jupiter, Vintage), the Arrange-Act-Assert pattern, assertions, parameterized tests, test lifecycle callbacks, Mockito test doubles, Spring Boot test slices (@WebMvcTest, @DataJpaTest, @SpringBootTest), Testcontainers integration with real databases, flaky test remediation, 30 rigorous interview questions with detailed answers, and production engineering checklists.

Table of Contents

PART I: FUNDAMENTALS & UNIT TESTING (Chunk 1 of 2)

1. What Is JUnit 5? (JUnit Platform, Jupiter, Vintage Architecture) Section 1
2. What Is a Unit Test? (Arrange–Act–Assert AAA Pattern) Section 2
3. Why Unit Tests Matter (Regression Prevention & Living Documentation) Section 3
4. Your First JUnit 5 Test (Maven Project Structure & Visibility Rules) Section 4
5. Assertions: Standard Ordering, Verification & assertAll Grouping Section 5
6. Test Names Should Describe Behavior (method_condition_expectedResult) Section 6
7. Build a Realistic Unit-Tested Class (PriceCalculator with Domain Contracts) Section 7
8. Test Exceptions (assertThrows, Exception Types & Message Inspection) Section 8
9. Parameterized Tests (@ParameterizedTest, @CsvSource, @ValueSource) Section 9
10. Understand the Test Lifecycle (@BeforeAll, @BeforeEach, @AfterEach, @AfterAll) Section 10
11. Test Collections and Returned Data (Size, Presence & Order Semantics) Section 11
12. Mocking Collaborators with Mockito (@Mock, when, thenReturn, verify) Section 12
13. Run Tests with Maven (Surefire Reports & Filtered Test Execution) Section 13
14. Exercise: Test the Calculator (Complete Matrix of Cases) Section 14
15. F.I.R.S.T. Unit Test Reliability Principles Section 15
16. Unit Tests versus Integration Tests (Key Architectural Differences) Section 16
17. Test Behavior, Not Implementation Details (Avoiding Brittle Tests) Section 17
18. Common Testing Failure Scenarios (Triage & Root Causes) Section 18
19. Interview Questions — 15 Core Fundamentals Questions & Answers Section 19
20. Hands-on Mastery Checklist Section 20

PART II: ADVANCED TESTING & PRODUCTION MASTERY (Chunk 2 of 2)

1. The Testing Pyramid (Unit vs. Integration vs. End-to-End Distribution) Section 1
2. What Makes a Good Test? (Deterministic, Isolated, Readable, Behavior-focused) Section 2
3. Test Doubles: Dummy, Stub, Mock, Fake, and Spy (Trade-offs & Pitfalls) Section 3
4. Spring Boot Testing Annotations (@WebMvcTest, @DataJpaTest, @SpringBootTest) Section 4
5. Unit-Test a Spring Service with Mockito (Constructor Injection & Fail-Fast) Section 5
6. Test a Spring MVC Controller (@WebMvcTest, MockMvc, MockitoBean) Section 6
7. Test a Repository Against a Database (@DataJpaTest, flush, clear, Testcontainers) Section 7
8. Test Configuration & External Dependencies (@DynamicPropertySource, Test Profiles) Section 8
9. Maven Testing Workflow (Surefire vs. Failsafe & Triage Sequences) Section 9
10. Flaky Tests and CI Reliability (Awaitility, Clocks & Mutable State Resets) Section 10
11. Common Production Testing Failures (Local vs. CI & Context Leakage) Section 11
12. Testing Security & Failure Paths (401, 403, RFC 7807 & Idempotency) Section 12
13. Advanced Interview Questions — 15 Architectural & Production Scenarios Section 13

14. Hands-On Capstone Lab & Self-Assessment Mastery Check Section 14
15. JUnit 5 Complete Mastery Checklist & Final Protocol Section 15

PART I: FUNDAMENTALS & UNIT TESTING

Phase I — Developer Foundations | Chunk 1 of 2

JUnit 5 — Chunk 1: Fundamentals & Unit Testing

Backend Engineering Tools & Infrastructure Handbook

Phase: I — Developer Foundations

Chunk: 1 of 2

Source file: `content/junit5/01-junit5-fundamentals-unit-testing.md`

Learning Objectives

By the end of this chapter, you should be able to:

- Explain unit testing and the role of JUnit 5.
- Write tests for Java classes and business logic.
- Use assertions, lifecycle methods, parameterized tests and exception testing.
- Separate unit tests from integration tests.
- Structure tests for readable, reliable feedback.
- Run and troubleshoot tests through Maven.
- Explain fundamental testing concepts in SDE-1 interviews.

Layer 1 — Fundamentals & Mental Models

1. What is JUnit 5?

Imagine you're developing a Java backend with an order-pricing service. It calculates totals, validates quantities and applies discounts.

You could test every change manually by starting the application and submitting requests. But that becomes slow and unreliable as the project grows.

JUnit is a testing framework for Java. JUnit 5 is the modern generation built around the JUnit Platform, JUnit Jupiter and JUnit Vintage.

The JUnit 5 Modular Architecture

```
+-------------------------------------------------------------------------------+
|                                JUnit 5                                        |
+-------------------------------------------------------------------------------+
|                              JUnit Platform                                   |
|           Foundation for launching test engines on the JVM / build tools      |
+--------------------------------------+----------------------------------------+
|            JUnit Jupiter             |             JUnit Vintage              |
|   New programming & extension model  |   Backward compatibility engine for    |
|   for writing JUnit 5 tests & runners |       running JUnit 3 & JUnit 4        |
+--------------------------------------+----------------------------------------+
```

- **JUnit Platform:** infrastructure for launching and integrating test engines with build tools (Maven Surefire, Gradle) and IDEs (IntelliJ, Eclipse, VS Code).
- **JUnit Jupiter:** the programming model and extension support for writing modern JUnit 5 tests (`org.junit.jupiter.api`).
- **JUnit Vintage:** supports running legacy JUnit 3 and JUnit 4 tests on the modern JUnit Platform.

For modern Java backends, you will primarily write tests using JUnit Jupiter.

---

2. What is a unit test?

A unit test checks a small piece of behavior in isolation, commonly a single method or class.

Suppose you have this Java class:

```
package com.example.pricing;

public class PriceCalculator {

    public int total(int unitPrice, int quantity) {
        return unitPrice * quantity;
    }
}
```

A unit test checks whether it produces the expected result:

```
package com.example.pricing;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

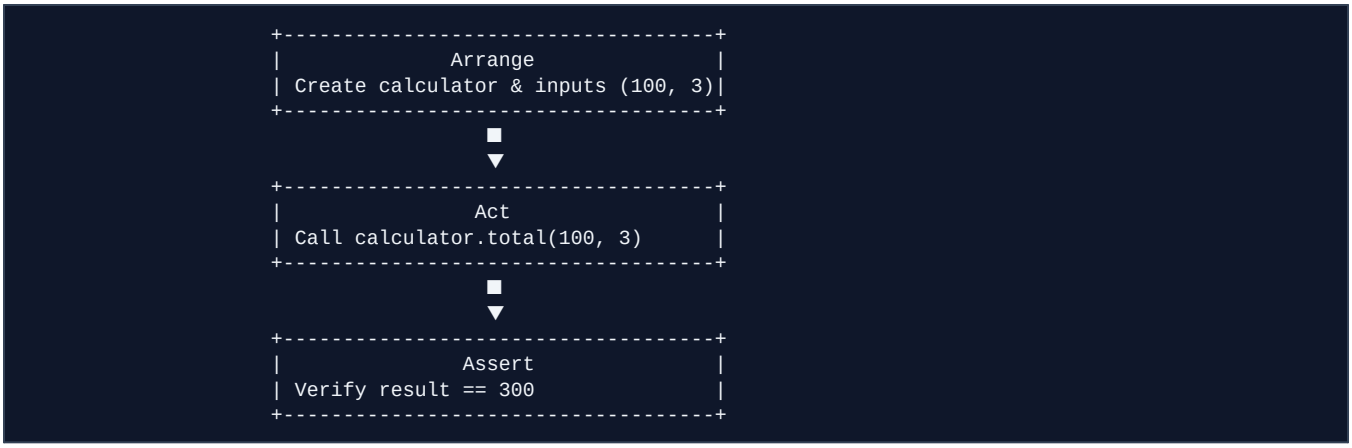
class PriceCalculatorTest {

    @Test
    void calculatesTotalPrice() {
        PriceCalculator calculator = new PriceCalculator();

        int result = calculator.total(100, 3);

        assertEquals(300, result);
    }
}
```

The AAA (Arrange–Act–Assert) Pattern



1. **Arrange:** set up the preconditions, instantiate objects, and prepare input fixtures.
2. **Act:** invoke the specific method or behavior under test.
3. **Assert:** verify that the observed outcome matches the expected specification.

A good test communicates the intended business behavior, not merely the mechanics of calling a method.

---

3. Why unit tests matter

Unit tests are fundamental to production engineering:

- **Regression safety:** catch unintended side effects immediately when code is modified.
- **Executable documentation:** serve as living, authoritative specifications of how classes are designed to behave.
- **Refactoring confidence:** allow restructuring internal algorithms without fear of breaking contracts.
- **Edge case verification:** systematically exercise nulls, negative numbers, overflow, and empty collections.
- **Fast feedback loop:** execute thousands of assertions in seconds without booting heavyweight servers or databases.
- **Automated CI quality gates:** block defective pull requests from reaching production environments.

> ***Production Lesson:*** *Tests do not prove that software is 100% defect-free. They provide verifiable evidence that the tested behavior behaves correctly for the scenarios and boundaries covered.*

---

4. Your first JUnit 5 test

In a standard Maven project, place source and test files strictly according to convention:

```
orders-service/
■■■ pom.xml
■■■ src/
    ■■■ main/
    ■   ■■■ java/
    ■       ■■■ com/example/pricing/PriceCalculator.java
    ■■■ test/
        ■■■ java/
            ■■■ com/example/pricing/PriceCalculatorTest.java
```

Maven Dependency Configuration

For Spring Boot projects, the starter test bundle imports JUnit 5 automatically:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

For plain Java applications without Spring Boot:

```
<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.10.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

The fundamental annotation is:

```
@Test
void shouldCalculateTotal() {
    // test behavior
}
```

Notice that in JUnit 5, test classes and test methods do **not** need to be declared `public`. Package-private visibility is preferred and idiomatic.

---

5. Assertions: how tests decide whether behavior is correct

Assertions compare actual runtime outcomes with expected values:

```
import static org.junit.jupiter.api.Assertions.*;

assertEquals(10, actual);
assertNotEquals(0, actual);
assertTrue(actual > 0);
assertFalse(items.isEmpty());
assertNull(value);
assertNotNull(value);
assertSame(expectedRef, actualRef);
assertNotSame(obj1, obj2);
```

Ordering Convention

Always follow the standard parameter order:

`assertEquals(expected, actual, optionalMessage);`

Reversing expected and actual produces confusing diagnostic failure messages in CI logs:

```
// Incorrect: assertEquals(actual, 300);
// Failure message: "Expected: <300>, but was: <actual>" (misleading!)
```

Multiple Assertions with `assertAll`

When multiple assertions are placed sequentially, the first failure immediately terminates test execution, masking subsequent failures:

```
// If id fails, status and quantity are never evaluated!
assertEquals("order-123", order.id());
assertEquals("CREATED", order.status());
assertEquals(3, order.quantity());
```

Use `assertAll` to group assertions so that all failures are reported together:

```
assertAll("Order attributes validation",
    () -> assertEquals("order-123", order.id(), "Incorrect order ID"),
    () -> assertEquals("CREATED", order.status(), "Incorrect status"),
    () -> assertEquals(3, order.quantity(), "Incorrect quantity")
);
```

---

6. Test names should describe behavior

Compare these naming styles:

Poor Test Name	Good Behavioral Test Name	Rationale
`test1()`	`calculatesTotalForPositiveQuantity()`	Explicitly documents the scenario being validated
`testCalculate()`	`total_withNegativeUnitPrice_throwsIllegalArgumentException()`	Follows `method_condition_expectedResult`
`checkOrder()`	`createOrder_whenInventoryIsInsufficient_rejectsOrder()`	Documents business domain rule

A standard convention:

`methodUnderScope_conditionUnderObservation_expectedOutcome`

Consistency across the engineering team matters more than any single dogmatic naming style.

Layer 2 — Practical Engineering & Workflows

7. Build a realistic unit-tested class

Let's make the price calculator robust against invalid domain states and arithmetic errors:

```
package com.example.pricing;

public class PriceCalculator {

    public int total(int unitPrice, int quantity) {
        if (unitPrice < 0) {
            throw new IllegalArgumentException("Unit price cannot be negative");
        }

        if (quantity <= 0) {
            throw new IllegalArgumentException("Quantity must be positive");
        }

        return Math.multiplyExact(unitPrice, quantity);
    }
}
```

This class defines four explicit contracts:

1. Valid inputs produce the correct total (`unitPrice * quantity`).
2. Negative unit prices are rejected with `IllegalArgumentException`.
3. Zero or negative quantities are rejected with `IllegalArgumentException`.
4. Large values that overflow standard 32-bit integer limits throw `ArithmeticException`.

---

8. Test exceptions

Use `assertThrows` to verify expected exception types:

```
package com.example.pricing;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

class PriceCalculatorTest {

    private final PriceCalculator calculator = new PriceCalculator();

    @Test
    void rejectsNegativeUnitPrice() {
        var exception = assertThrows(
            IllegalArgumentException.class,
            () -> calculator.total(-100, 2)
        );

        assertEquals("Unit price cannot be negative", exception.getMessage());
    }

    @Test
    void rejectsZeroQuantity() {
        var exception = assertThrows(
            IllegalArgumentException.class,
            () -> calculator.total(100, 0)
        );

        assertEquals("Quantity must be positive", exception.getMessage());
    }

    @Test
    void throwsArithmeticExceptionOnIntegerOverflow() {
        assertThrows(
            ArithmeticException.class,
            () -> calculator.total(Integer.MAX_VALUE, 2)
        );
    }
}
```

If the code inside the lambda completes without throwing an exception, or throws an unrelated exception type, the test fails immediately.

---

9. Test multiple input scenarios with parameterized tests

Parameterized tests run the same test logic against a table of distinct inputs and expected outputs, eliminating repetitive boilerplate test methods.

Add the dependency (included in `spring-boot-starter-test`):

```
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-params</artifactId>
    <scope>test</scope>
</dependency>
```

Parameterized Test with `@CsvSource`

```
package com.example.pricing;

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.ValueSource;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

class PriceCalculatorParameterizedTest {

    private final PriceCalculator calculator = new PriceCalculator();

    @ParameterizedTest(name = "Price: {0}, Qty: {1} => Total: {2}")
    @CsvSource({
        "100, 1, 100",
        "100, 3, 300",
        "250, 4, 1000",
        "0,   5, 0"
    })
    void calculatesTotalAcrossValidInputs(int unitPrice, int quantity, int expected) {
        assertEquals(expected, calculator.total(unitPrice, quantity));
    }

    @ParameterizedTest
    @ValueSource(ints = {0, -1, -10, -500})
    void rejectsInvalidQuantities(int invalidQuantity) {
        assertThrows(
            IllegalArgumentException.class,
            () -> calculator.total(100, invalidQuantity)
        );
    }
}
```

10. Understand the test lifecycle

JUnit Jupiter manages the execution lifecycle of each test class:

Annotation	Execution Timing	Default Visibility / Method Signature
`@BeforeAll`	Runs once before any tests in the class	Must be `static void` (unless `PER_CLASS`)
`@BeforeEach`	Runs before every individual `@Test`	`void` (runs per instance)
`@Test`	The test method execution	`void`
`@AfterEach`	Runs after every individual `@Test`	`void` (cleanup per instance)
`@AfterAll`	Runs once after all tests in the class finish	Must be `static void` (unless `PER_CLASS`)

Example Lifecycle Implementation

```
package com.example.state;

import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.assertEquals;

class CounterTest {

    private Counter counter;

    @BeforeAll
    static void initGlobalFixtures() {
        // Expensive one-time initialization (e.g. static configuration)
    }

    @BeforeEach
    void setUp() {
        // Fresh instance created before each test to prevent cross-test pollution
        counter = new Counter();
    }

    @AfterEach
    void tearDown() {
        // Teardown per-test state
    }

    @Test
    void startsAtZero() {
        assertEquals(0, counter.value());
    }

    @Test
    void incrementsByOne() {
        counter.increment();
        assertEquals(1, counter.value());
    }
}
```

> ***Crucial Rule:*** *By default, JUnit creates a* ***new test-class instance*** *for every single test method. This design guarantees test isolation by preventing field modifications in one test from leaking into another.*

---

11. Test collections and returned data

When testing lists, maps, or sets, verify collection semantics rather than just checking if the object is non-null:

```
List<String> items = catalog.findProductsByCategory("Electronics");

assertAll("Product list validations",
    () -> assertNotNull(items),
    () -> assertEquals(2, items.size(), "Unexpected element count"),
    () -> assertTrue(items.contains("Laptop")),
    () -> assertTrue(items.contains("Mouse")),
    () -> assertFalse(items.contains("Apple"))
);
```

Ensure you verify:

- Expected size and presence of key items.
- Correct ordering when sorting is guaranteed by the API contract.
- Empty collection handling (`assertTrue(list.isEmpty())`).
- Avoid fragile assertions that fail merely if elements appear in an unspecified order.

12. Mocking: isolate the unit under test

A pure unit test tests one class in complete isolation from external databases, networks, and services.

Consider `OrderService`:

```
package com.example.orders;

import java.util.Optional;

public class OrderService {

    private final OrderRepository repository;

    public OrderService(OrderRepository repository) {
        this.repository = repository;
    }

    public Order findById(String id) {
        return repository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("Order not found: " + id));
    }
}
```

Using Mockito, we mock `OrderRepository` so no actual database or connection pool is involved:

```
package com.example.orders;

import org.junit.jupiter.api.Test;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.mockito.Mockito.*;

class OrderServiceTest {

    @Test
    void returnsOrderWhenPresentInRepository() {
        // 1. Arrange: Create mock collaborator and stub behavior
        OrderRepository repository = mock(OrderRepository.class);
        Order expectedOrder = new Order("ord-101", "PAID");

        when(repository.findById("ord-101"))
            .thenReturn(Optional.of(expectedOrder));

        OrderService service = new OrderService(repository);

        // 2. Act
        Order actualOrder = service.findById("ord-101");

        // 3. Assert: Verify returned entity and interaction
        assertEquals(expectedOrder, actualOrder);
        verify(repository, times(1)).findById("ord-101");
    }

    @Test
    void throwsExceptionWhenOrderNotFound() {
        OrderRepository repository = mock(OrderRepository.class);

        when(repository.findById("missing-id"))
            .thenReturn(Optional.empty());

        OrderService service = new OrderService(repository);

        assertThrows(
            IllegalArgumentException.class,
            () -> service.findById("missing-id")
        );
    }
}
```

When to Mock vs. When Not to Mock:

- **Mock**: Database repositories, third-party payment gateways, SMS/email senders, external microservice clients.
- **Do Not Mock**: Pure domain entities, value objects, simple DTOs, standard Java collections (`List`, `Map`), or the class under test itself.

---

13. Run tests with Maven

Execute the test phase via the Maven Wrapper:

```
./mvnw test
```

Run a specific test class:

```
./mvnw test -Dtest=PriceCalculatorTest
```

Run a single test method:

```
./mvnw test -Dtest=PriceCalculatorTest#calculatesTotalPrice
```

Execute full verification (compilation, unit tests, packaging, integration tests):

```
./mvnw clean verify
```

If tests fail, inspect the console error trace or view the detailed Surefire XML and text reports under:

```
target/surefire-reports/
```

---

14. Exercise: test the calculator

Verify your understanding by implementing the complete matrix of calculator test cases:

Case	Inputs	Expected Outcome
Normal calculation	`unitPrice=100`, `quantity=2`	Returns `200`
Identity quantity	`unitPrice=100`, `quantity=1`	Returns `100`
Zero unit price	`unitPrice=0`, `quantity=5`	Returns `0`
Negative unit price	`unitPrice=-10`, `quantity=2`	Throws `IllegalArgumentException`
Zero quantity	`unitPrice=100`, `quantity=0`	Throws `IllegalArgumentException`
Negative quantity	`unitPrice=100`, `quantity=-3`	Throws `IllegalArgumentException`
Multiplication overflow	`unitPrice=Integer.MAX_VALUE`, `quantity=2`	Throws `ArithmeticException`

Layer 3 — Production Engineering & Interview Mastery

15. What makes a unit test reliable?

A production-grade unit test adheres to the **F.I.R.S.T.** principles:

- **Fast:** Executes in milliseconds. If unit tests take minutes, engineers stop running them locally.
- **Isolated / Independent:** Tests never depend on execution order or side effects from prior tests.
- **Repeatable / Deterministic:** Returns identical results regardless of whether executed on macOS, Linux CI, or at midnight on New Year's Eve.
- **Self-Validating:** Evaluates to a boolean pass or fail; no manual log inspection required.
- **Timely / Thorough:** Written concurrently with production code, covering happy paths, edge boundaries, and exception branches.

Avoid anti-patterns:

- Flaky tests caused by `Thread.sleep()`.
- Tests that depend on current machine timezone or default locale.
- Tests that access external network URLs.

---

16. Unit tests versus integration tests

Dimension	Unit Tests	Integration Tests
Scope	Single class or isolated method	Multiple collaborating components
Dependencies	Mocks or stubs for all external I/O	Real databases (PostgreSQL/H2), Spring context
Speed	1–5 milliseconds per test	500ms to several seconds per test
Failure Isolation	Pinpoints the exact line in a class	Failure could be in DB, query, mapping, or network
Maven Plugin	`maven-surefire-plugin` (`mvn test`)	`maven-failsafe-plugin` (`mvn verify`)
Naming Convention	`*Test.java`	`*IT.java` or `*IntegrationTest.java`

A resilient backend uses both: high volumes of fast unit tests for complex business algorithms, supplemented by targeted integration tests verifying persistence and HTTP wiring.

---

17. Test behavior, not implementation details

Avoid **overspecified tests** that assert every internal private helper call or intermediate variable:

```
// Fragile / Brittle: Couples test to internal private method names
verify(service, times(1)).doInternalHelperStepA();
verify(service, times(1)).doInternalHelperStepB();
```

If you refactor internal algorithms without changing the external business contract, existing unit tests should continue passing without modification. Test inputs and observable outputs, not private implementation plumbing.

---

18. Common failure scenarios

Scenario A: Tests pass individually but fail when run together

- **Root Cause**: Leaked static state or shared mutable singleton modified during one test, contaminating downstream tests.
- **Remediation**: Avoid static mutable variables; reset shared state in `@BeforeEach` / `@AfterEach`.

Scenario B: Tests pass locally but fail in CI

- **Root Cause**: Discrepancies in JDK versions, APFS (case-insensitive) vs. Linux ext4 (case-sensitive) file lookups, or unconfigured locale/timezone differences.

- **Remediation**: Run tests in Docker with identical Linux JDK configurations; isolate date logic with fixed `Clock` instances (`Clock.fixed()`).

Scenario C: A unit test takes several seconds to execute

- **Root Cause**: Accidentally booting the full Spring application context (`@SpringBootTest`) instead of running a pure Mockito unit test (`@ExtendWith(MockitoExtension.class)`).
- **Remediation**: Strip Spring annotations from business unit tests.

Scenario D: High test coverage but production bugs escape

- **Root Cause**: Shallow assertion syndrome—tests exercise code paths to increase metrics but perform no assertions or assert only non-nullness (`assertNotNull(res)`).
- **Remediation**: Enforce strict assertion reviews and mutation testing (e.g. PIT).

---

19. Interview questions — 15 essentials

Q1. What is JUnit 5 and how is it architected?

Answer: JUnit 5 is the modern generation of the Java testing framework. It is decoupled into three distinct sub-projects: **JUnit Platform** (runtime foundation for launching test engines), **JUnit Jupiter** (the programming model and extension API for writing JUnit 5 tests), and **JUnit Vintage** (backward compatibility engine for running JUnit 3/4 tests).

Q2. What does the `@Test` annotation signify in JUnit 5?

Answer: It marks a method as a test method to be discovered and executed by the JUnit Jupiter engine. In JUnit 5, test methods and classes do not need to be `public`; package-private visibility is preferred.

Q3. Explain the Arrange–Act–Assert (AAA) pattern.

Answer: A standard structural pattern for unit tests:

1. **Arrange**: Set up fixtures, inputs, and mock collaborator responses.
2. **Act**: Invoke the method under test with the prepared inputs.
3. **Assert**: Verify that the return value, exception, or state mutation matches the expected specification.

Q4. What is the difference between `assertEquals` and `assertSame`?

Answer: `assertEquals` verifies logical equality using the target object's `equals(Object)` method. `assertSame` checks memory reference equality using the `==` operator to confirm both references point to the exact same heap instance.

Q5. How do you test that a method throws an expected exception in JUnit 5?

Answer: Use `assertThrows(ExpectedException.class, () -> methodCall())`. It returns the thrown exception instance, allowing further assertions on the error message or custom exception fields.

Q6. What is the difference between `@BeforeEach` and `@BeforeAll`?

Answer: `@BeforeEach` executes before every individual `@Test` method in the class, operating on that test's fresh instance. `@BeforeAll` executes once before all test methods in the class begin; it must be declared `static` under the default test lifecycle.

Q7. What are parameterized tests and why are they valuable?

Answer: Tests annotated with `@ParameterizedTest` that execute the same test logic repeatedly across multiple arguments supplied by sources like `@ValueSource`, `@CsvSource`, or `@MethodSource`. They eliminate duplicated test boilerplate and simplify boundary-value analysis.

Q8. What is mocking and why is it used in unit testing?

Answer: Mocking is the practice of replacing real external collaborators (databases, third-party HTTP clients, messaging brokers) with simulated test doubles that return controlled stub responses and record method invocations, enabling fast, isolated verification of business logic.

Q9. When should you use a mock versus a real object?

Answer: Use mocks for slow, non-deterministic, or out-of-process collaborators (databases, network APIs, disk I/O, email servers). Use real objects for domain entities, value objects, mathematical utilities, and pure data structures.

Q10. What is the fundamental difference between a unit test and an integration test?

Answer: A **unit test** validates a single class or method in complete isolation without external I/O or frameworks, running in milliseconds. An **integration test** verifies the interaction and wiring across multiple layers (e.g. Spring controllers, database repositories, JPA mappings) against real or containerized infrastructure.

Q11. What causes flaky tests and how do you prevent them?

Answer: Flaky tests are caused by non-deterministic factors: reliance on real system clocks, `Thread.sleep()`, race conditions in concurrent code, dependency on test execution order, unseeded random number generators, or network latency. Prevent them by using deterministic clocks (`Clock.fixed()`), dependency injection, and clean state resets.

Q12. Why does JUnit create a new instance of the test class for each test method?

Answer: To enforce test isolation. By instantiating a fresh class instance for every `@Test`, instance variables modified in one test cannot pollute or influence subsequent test outcomes.

Q13. How are JUnit 5 tests executed in a Maven build?

Answer: Maven Surefire Plugin discovers and runs test classes ending in `*Test`, `Test*`, or `*TestCase` during the `test` phase (`mvn test`), reporting results in `target/surefire-reports/`.

Q14. What is the benefit of `assertAll` over multiple sequential assertions?

Answer: Sequential assertions fail-fast on the first mismatch, hiding information about whether remaining assertions would have passed or failed. `assertAll` evaluates every supplied assertion independently and reports all failures together in a unified error summary.

Q15. Does 100% code coverage mean software is bug-free?

Answer: No. Code coverage only measures which lines of bytecode were executed during testing. It does not prove that assertions were meaningful, that boundary edge cases were evaluated, or that specifications were correct.

---

20. Hands-on mastery checklist

- [ ] Create a Maven project with JUnit Jupiter configured.
- [ ] Write a passing unit test using `@Test` and `assertEquals`.
- [ ] Test invalid input and boundary exceptions using `assertThrows`.
- [ ] Create a parameterized test with `@CsvSource` and `@ValueSource`.
- [ ] Use `@BeforeEach` to initialize clean test state.
- [ ] Mock a repository and test service behavior using Mockito (`when`, `thenReturn`, `verify`).
- [ ] Run tests through Maven (`./mvnw test`) and inspect Surefire reports.
- [ ] Explain the difference between unit and integration tests.
- [ ] Identify one possible cause of a flaky test and write a deterministic fix.
- [ ] Add tests for boundaries, arithmetic overflow, and failure scenarios.

Completion Protocol

1. Authoritative source file: `content/junit5/01-junit5-fundamentals-unit-testing.md`.
2. Coverage of all three layers: Fundamentals & Mental Models, Practical Engineering & Workflows, and Production Engineering & Interview Mastery.
3. No chunk-level PDF generated.
4. Next chunk: `content/junit5/02-junit5-advanced-testing-production.md`.
5. Compile `docs/junit5/BEH_JUNIT5_COMPLETE_HANDBOOK.pdf` only after Chunk 2 is complete.
6. Commit with exact message `JUnit 5 done` only upon complete tool verification.

JUnit 5 progress: 1 of 2 educational chunks authored.

PART II: ADVANCED TESTING & PRODUCTION MASTERY

Phase I — Developer Foundations | Chunk 2 of 2

JUnit 5 — Chunk 2: Advanced Testing & Production Mastery

Backend Engineering Tools & Infrastructure Handbook

Phase: I — Developer Foundations

Chunk: 2 of 2

Source file: `content/junit5/02-junit5-advanced-testing-production.md`

Learning Objectives

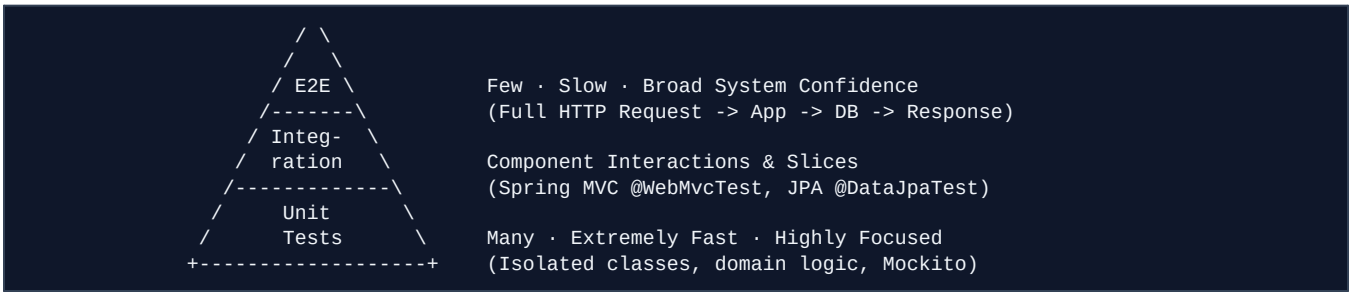
By the end of this chapter, you should be able to:

- Design unit, integration, and Spring Boot tests with clear boundaries.
- Choose correctly between mocks, stubs, fakes, and real dependencies.
- Test service and repository behavior without creating brittle tests.
- Diagnose flaky tests and failures in CI.
- Explain testing trade-offs in backend engineering interviews.

Layer 1 — Fundamentals & Mental Models

1. The Testing Pyramid

Backend systems have multiple testing layers. Each answers a different question.



> ***Architectural Principle:*** *The testing pyramid is a practical mental model, not a dogmatic mathematical formula. The optimal distribution depends on your application architecture, domain complexity, and failure risks.*

Unit tests

Test a small unit of behavior in isolation, usually a method or class.

Example: Does `OrderService` reject an order when inventory is insufficient?

- Sub-millisecond execution.
- Dependencies replaced with lightweight test doubles (mocks/stubs).
- Failures pinpoint the exact line of code.

Integration tests

Test whether components work together correctly across architectural boundaries.

Examples:

- Spring Data repository queries execute against a real PostgreSQL container.
- A Spring controller correctly maps URLs, deserializes JSON, and enforces security filters.
- Transaction boundaries rollback correctly upon unhandled runtime exceptions.
- A cache service serializes keys and values correctly to Redis.

Integration tests catch wiring, serialization, dialect, and configuration defects that isolated unit tests cannot see.

End-to-end (E2E) tests

Exercise a complete user journey through the entire application stack, from external HTTP request to database persistence and downstream events.

They provide broad confidence but are slower to execute, more complex to maintain, and produce failures that are harder to localize.

Production Rule: Test business algorithms primarily at the unit level, architectural boundaries with integration tests, and a small number of critical customer journeys end-to-end.

---

2. What Makes a Good Test?

A reliable test possesses four fundamental properties:

1. **Deterministic:** The same code and input conditions produce the exact same outcome on every run.
2. **Isolated:** Unrelated tests do not share mutable state, execute in arbitrary order, or contaminate the environment.
3. **Readable:** An engineer reading the test can immediately understand the business rule or contract being validated.
4. **Behavior-focused:** It validates observable contract specifications rather than private internal implementation details.

Consider an order service:

A **behavior-focused test** asks:

> *Given sufficient inventory, does the service create the order, decrement stock, and return the generated order identifier?*

A **brittle test** asks:

> *Did the service call private helper `doInternalCalculations()` exactly twice in sequence?*

Testing internal plumbing makes tests fragile during refactoring, even when user-facing behavior remains completely unchanged. Test observable contracts, not implementation details.

---

3. Test Doubles: Mock, Stub, Fake, and Spy

Gerard Meszaros defined the standard taxonomy of **Test Doubles**:

Type	Formal Definition	Typical Production Use Case
Dummy	Passed around but never actually used	Supplying non-null placeholder parameters
Stub	Returns hardcoded, predetermined values to method calls	Inventory client configured to return `10` available units
Mock	Pre-programmed with expectations; verifies interactions	Verifying that an email notification service was invoked with specific payload
Fake	Simplified working implementation with shortcut logic	In-memory `ConcurrentHashMap` repository instead of PostgreSQL
Spy	Wraps a real object; delegates calls while recording interactions	Spying on a real Spring service to verify invocation metrics

The Critical Architectural Trade-Off

Mocks are invaluable when a collaborator is slow, expensive, nondeterministic, or out-of-process (e.g. third-party Stripe API, SMS gateway).

However, **over-mocking creates a false sense of security**:

- Mocking a database repository does not prove your JPQL or native SQL query syntax is valid.
- Mocking an HTTP client does not prove Jackson can deserialize the real third-party JSON payload.

Interview Principle: Use test doubles to isolate unit business logic, but use real implementations (via Testcontainers or test slices) where integration correctness is what must be verified.

---

4. Spring Boot Testing Annotations

Spring Boot provides specialized **Test Slices** that configure only the specific subset of beans needed for a test, dramatically improving speed.

Annotation	Loaded Context Scope	Typical Target Layer
`@ExtendWith(MockitoExtension.class)`	No Spring context; pure JUnit 5 + Mockito	Pure service/domain business unit tests
`@SpringBootTest`	Full Spring application context with all beans	Full integration tests and end-to-end flows
`@WebMvcTest`	Spring MVC layer only (Controllers, Filters, Formatters)	HTTP routes, status codes, JSON validation
`@DataJpaTest`	JPA repositories, entities, EntityManager	Database queries, mappings, transaction rollbacks

Annotation	Loaded Context Scope	Typical Target Layer
`@ActiveProfiles("test")`	Activates `application-test.yml`	Overriding database URLs, secrets, or ports

Anti-Patterns to Avoid:

1. **Using `@SpringBootTest` everywhere:** Boots the entire application context (databases, caches, security, scheduled jobs), slowing the test suite down to minutes.
2. **Assuming `@WebMvcTest` boots services:** It only instantiates controllers. Service collaborators must be explicitly provided as mocks (`@MockBean` or `@MockitoBean`).

Layer 2 — Practical Engineering & Workflows

5. Unit-Test a Spring Service with Mockito

Consider an order placement workflow:

Production Interfaces & Service

```
package com.example.orders;

public interface InventoryClient {
    int availableUnits(String productId);
}
```

```
package com.example.orders;

public interface OrderRepository {
    Order save(Order order);
}
```

```
package com.example.orders;

public record Order(String productId, int quantity) {}
```

```
package com.example.orders;

public class OrderService {
    private final InventoryClient inventory;
    private final OrderRepository orders;

    public OrderService(InventoryClient inventory, OrderRepository orders) {
        this.inventory = inventory;
        this.orders = orders;
    }

    public Order placeOrder(String productId, int quantity) {
        if (quantity <= 0) {
            throw new IllegalArgumentException("Quantity must be positive");
        }

        if (inventory.availableUnits(productId) < quantity) {
            throw new IllegalStateException("Insufficient inventory");
        }

        return orders.save(new Order(productId, quantity));
    }
}
```

Complete Unit Test with Mockito

```
package com.example.orders;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock
    InventoryClient inventory;

    @Mock
    OrderRepository orders;

    OrderService service;

    @BeforeEach
    void setUp() {
        service = new OrderService(inventory, orders);
    }

    @Test
    void placesOrderWhenInventoryIsSufficient() {
        // Arrange
        when(inventory.availableUnits("P-101")).thenReturn(10);
        Order expected = new Order("P-101", 2);
        when(orders.save(any(Order.class))).thenReturn(expected);

        // Act
        Order result = service.placeOrder("P-101", 2);

        // Assert
        assertEquals(expected, result);
        verify(orders).save(new Order("P-101", 2));
    }

    @Test
    void rejectsOrderWhenInventoryIsInsufficient() {
        // Arrange
        when(inventory.availableUnits("P-101")).thenReturn(1);

        // Act & Assert
        assertThrows(
            IllegalStateException.class,
            () -> service.placeOrder("P-101", 2)
        );

        verify(orders, never()).save(any());
    }

    @Test
    void rejectsNonPositiveQuantityBeforeCheckingInventory() {
        // Act & Assert
        assertThrows(
            IllegalArgumentException.class,
            () -> service.placeOrder("P-101", 0)
        );

        // Verify fail-fast behavior: inventory is never contacted
        verifyNoInteractions(inventory, orders);
    }
}
```

Key Engineering Practices:

- **Constructor Injection:** Allows direct instantiation with mock doubles without booting a Spring container.
- **Fail-Fast Verification (`verifyNoInteractions`):** Proves that invalid inputs are rejected before triggering expensive remote inventory calls.
- **`never().save(any())`:** Confirms database writes are prevented when business validations fail.

---

6. Test a Spring MVC Controller with `@WebMvcTest`

Test HTTP routing, JSON serialization, and error handling without starting a live web server:

```
package com.example.orders;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.test.context.bean.override.mockito.MockitoBean; // Spring Boo
t 3.4+
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath
;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@WebMvcTest(OrderController.class)
class OrderControllerTest {

    @Autowired
    MockMvc mockMvc;

    @MockitoBean // Or @MockBean in earlier Spring Boot versions
    OrderService orderService;

    @Test
    void returnsOrderAsJson() throws Exception {
        when(orderService.findById("O-101"))
            .thenReturn(new OrderResponse("O-101", "P-101"));

        mockMvc.perform(get("/orders/O-101"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value("O-101"))
            .andExpect(jsonPath("$.productId").value("P-101"));
    }
}
```

What this test verifies:

- HTTP GET mapping `/orders/{id}` routes to the controller method.
- Spring MVC serializes Java records/DTOs into JSON format.
- HTTP status code returns `200 OK`.
- Mockito mock isolates the controller from service and repository logic.

---

7. Test a Repository Against a Database with `@DataJpaTest`

Verify custom queries, JPA object-relational mappings, and unique constraints:

```
package com.example.orders;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.boot.test.autoconfigure.orm.jpa.TestEntityManager;

import static org.junit.jupiter.api.Assertions.assertEquals;

@DataJpaTest
class OrderEntityRepositoryTest {

    @Autowired
    TestEntityManager entityManager;

    @Autowired
    OrderEntityRepository repository;

    @Test
    void savesAndFindsOrderById() {
        OrderEntity entity = new OrderEntity();
        entity.setProductId("P-101");
        entity.setQuantity(2);

        OrderEntity saved = repository.save(entity);

        // Force SQL INSERT execution to database
        entityManager.flush();
        // Detach entities so subsequent findById queries actual database table
        entityManager.clear();

        OrderEntity found = repository.findById(saved.getId())
            .orElseThrow();

        assertEquals("P-101", found.getProductId());
        assertEquals(2, found.getQuantity());
    }
}
```

> ***Why `flush()` and `clear()` Matter:*** *By default, JPA caches entity instances in the first-level persistence context. Without* **`clear()`**, *calling* **`findById()`** *simply returns the cached object in RAM without executing any SQL SELECT against the database!*

Database Dialect Reality: Embedded H2 vs. Real PostgreSQL

Embedded databases (H2) often accept invalid SQL queries, ignore PostgreSQL JSONB operators, or have different case-sensitivity rules. If your production database is PostgreSQL, configure integration tests using **Testcontainers**.

---

8. Test Configuration and External Dependencies

```
@SpringBootTest
@Testcontainers
class OrderServicePostgresIT {

    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:16-alpi
ne");

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Test
    void verifiesRealPostgresPersistence() {
        // Runs against genuine disposable Dockerized PostgreSQL
    }
}
```

Production Rules:

1. **Never point tests at production or staging databases:** Tests modify, truncate, and corrupt state.
2. **Use `@DynamicPropertySource`:** Binds random dynamic container ports to Spring properties seamlessly.
3. **Isolate Test Profiles:** Store non-production defaults in `src/test/resources/application-test.yml`.

---

9. Maven Testing Workflow

Execute targeted testing workflows from the repository root:

```
# Run all unit tests
./mvnw test

# Clean target/ and run tests
./mvnw clean test

# Package artifact, including all verification phases
./mvnw package

# Run a specific test class
./mvnw -Dtest=OrderServiceTest test

# Run a specific test method
./mvnw -Dtest=OrderServiceTest#placesOrderWhenInventoryIsSufficient test

# Run integration tests bound to the verify phase (via maven-failsafe-plugin)
./mvnw verify
```

Triage Sequence for Failed Tests:

1. Read the **first assertion failure** message (downstream failures are frequently cascading side effects).
2. Distinguish compilation errors (`javac`) from runtime assertion errors (`AssertionFailedError`).
3. Inspect Surefire XML reports in `target/surefire-reports/`.
4. Run the failing test in isolation to rule out execution-order coupling.

Layer 3 — Production Engineering & Interview Mastery

10. Flaky Tests and CI Reliability

A **flaky test** exhibits non-deterministic behavior—passing and failing intermittently on the same code revision.

Top Causes of Test Flakiness:

1. **Arbitrary Sleeps (`Thread.sleep(1000)`):** Loaded CI runners with fewer vCPUs execute slower than developer MacBooks, triggering spurious timeouts.

Fix: Use **Awaitility** for bounded polling:

```
    await().atMost(5, SECONDS).until(() -> worker.isFinished());
```

2. **System Clock Reliance (`LocalDateTime.now()`):** Assertions fail when executed across midnight or in different server timezones (UTC vs. local).

Fix: Inject a fixed `Clock` instance:

```
    Clock fixedClock = Clock.fixed(Instant.parse("2026-10-23T08:00:00Z"), ZoneOffset.UTC)
;
```

3. **Shared Mutable State:** Static variables or shared database tables not cleared between tests.

Fix: Use `@BeforeEach` resets or `@DirtiesContext`.

4. **Order Dependency:** Test B only passes if Test A executed first to seed data.

Fix: Make every test self-sufficient and seed its own prerequisites.

---

11. Common Production Testing Failures

Scenario A — Works locally, fails in CI

- **Root Causes**: Different JDK versions, missing CI environment variables, case-sensitive Linux filesystem (APFS on macOS is case-insensitive), or timezone assumptions.
- **Action**: Inspect CI logs, verify `mvn -version`, enforce the Maven Wrapper (`./mvnw`), and run tests in Docker locally to reproduce.

Scenario B — Unit tests pass, API fails in production

- **Root Causes**: Incorrect Spring auto-configuration, JSON serialization/naming mismatch (`snake_case` vs `camelCase`), missing security filters, or database constraint violations.
- **Action**: Add a focused `@WebMvcTest` or Testcontainers integration test; do not turn every unit test into a full application test.

Scenario C — Tests pass individually, fail as a full suite

- **Root Causes**: Static state leakage, database records persisting across test classes, port collisions during parallel test execution.
- **Action**: Ensure test isolation; clean up resources in `@AfterEach`; verify `@DirtiesContext` usage for contaminated Spring contexts.

Scenario D — Test suite is painfully slow

- **Root Causes**: Booting multiple separate Spring contexts, excessive integration tests, or unnecessary real network latency.
- **Action**: Group tests under unified test configurations to allow Spring context caching, and shift business validation logic to pure Mockito unit tests.

---

12. Testing Security and Failure Paths

Robust production systems test error paths with the same rigor as happy paths:

- **Input Validation**: Assert `400 Bad Request` on malformed, boundary, or missing fields.
- **Authentication**: Assert `401 Unauthorized` when tokens are missing, expired, or tampered.
- **Authorization & RBAC**: Assert `403 Forbidden` when a standard user accesses an administrative route.
- **Resource Not Found**: Assert `404 Not Found` when requesting non-existent resource IDs.
- **Concurrency & Idempotency**: Assert that duplicate POST requests with the same idempotency key reject duplicate charges.
- **Information Leakage Prevention**: Verify that error payloads conform to RFC 7807 Problem Details and **never leak internal database stack traces or secrets**.

---

13. Fifteen Advanced Interview Questions

Q1. What is the difference between a unit test and an integration test?

Answer: A **unit test** isolates a small unit of behavior (a class or method), replacing all external dependencies with test doubles, running in milliseconds. An **integration test** verifies that multiple collaborating components (e.g. Spring controllers, repositories, databases, message brokers) work together correctly across architectural boundaries.

Q2. Why should you avoid using `@SpringBootTest` for every test?

Answer: `@SpringBootTest` boots the full Spring application context, instantiating all beans, configurations, and connections. Doing this for every test drastically inflates test execution time from seconds to minutes. Use focused unit tests or test slices (`@WebMvcTest`, `@DataJpaTest`) instead.

Q3. What is the difference between a mock and a stub?

Answer: A **stub** is an indirect input double that returns predetermined, hardcoded responses to method calls. A **mock** is an indirect output double that records incoming method invocations and allows the test to verify that specific interactions occurred.

Q4. What is the Arrange–Act–Assert (AAA) pattern?

Answer: A standard structural pattern for test readability:

1. **Arrange**: Set up fixtures, inputs, and stub expectations.
2. **Act**: Invoke the target method.
3. **Assert**: Verify return values, state mutations, and required interactions.

Q5. What does `assertThrows` do in JUnit 5?

Answer: It asserts that executing a supplied lambda throws a specific exception class (or subclass), failing the test if no exception or an incorrect exception is thrown, and returning the thrown exception instance for further assertions.

Q6. Why is mocking a repository insufficient to test a database query?

Answer: Mocking a repository stubs the Java interface without executing any SQL against a database engine. It cannot catch invalid SQL syntax, broken JPQL mappings, constraint violations, or database dialect differences.

Q7. What is a flaky test and why is it dangerous?

Answer: A flaky test passes and fails intermittently on identical code revisions due to timing, order dependence, or environmental factors. It erodes developer trust in CI pipelines, causing teams to ignore genuine regression failures.

Q8. How do you test asynchronous code reliably without `Thread.sleep`?

Answer: Use synchronization primitives (`CountDownLatch`, `CompletableFuture.join()`) or bounded polling libraries like **Awaitility** (`await().atMost(...).until(...)`) to await the actual completion condition.

Q9. What is `@DataJpaTest` used for?

Answer: It is a Spring Boot test slice that configures only JPA entities, Spring Data repositories, and an `EntityManager`, running each test in a transaction that is rolled back by default at test completion.

Q10. What is `@WebMvcTest` used for?

Answer: It is a Spring Boot test slice that configures only the presentation layer (Spring MVC controllers, filters, JSON serialization), allowing fast testing of HTTP status codes, routing, and payloads via `MockMvc` without starting an embedded servlet container.

Q11. Why can an embedded in-memory database (H2) hide production bugs?

Answer: H2 differs from production databases (like PostgreSQL) in supported SQL dialect, data types (JSONB, UUID), lock escalation models, constraint enforcement, and case sensitivity, allowing tests to pass that fail when deployed to production.

Q12. What is the purpose of Testcontainers?

Answer: Testcontainers provides lightweight, disposable Docker containers for integration tests, allowing tests to execute against genuine production-identical databases (PostgreSQL, MySQL), caches (Redis), and message brokers (Kafka).

Q13. Should a test verify every internal method invocation with `verify()`?

Answer: No. Overusing `verify()` couples tests to internal private implementation details rather than observable business contracts, creating brittle tests that break during harmless refactoring.

Q14. How do you systematically debug a test that passes locally but fails in CI?

Answer: Inspect CI console logs for the root cause; verify JDK vendor/version alignment; check for case-sensitive file path lookups; eliminate timezone/locale dependencies; and run tests in a clean Docker container matching the CI environment.

Q15. How do you balance test suite execution speed with production confidence?

Answer: Follow the testing pyramid: maintain a wide foundation of fast, deterministic unit tests for complex business rules; a focused set of slice/integration tests for database and API boundaries; and a minimal set of end-to-end smoke tests for critical user workflows.

---

14. Hands-On Capstone

Build an order processing service with three explicit requirements:

1. Reject non-positive quantities (`quantity <= 0`) with `IllegalArgumentException`.
2. Reject orders when inventory is insufficient with `IllegalStateException`.
3. Save valid orders and return the persisted result.

Implementation Checklist

- [ ] Write three isolated unit tests using JUnit 5 and Mockito (`@ExtendWith(MockitoExtension.class)`).
- [ ] Add a controller test with `@WebMvcTest` asserting HTTP `200 OK` and JSON response fields.
- [ ] Add a controller test asserting HTTP `400 Bad Request` on invalid input payloads.
- [ ] Add a repository integration test using `@DataJpaTest` with `TestEntityManager.flush()` and `clear()`.
- [ ] Run the complete test suite using `./mvnw clean test`.
- [ ] Confirm all tests pass independently and as a full suite.
- [ ] Introduce an intentional boundary error (e.g. change `<` to `<=`) and verify tests fail.
- [ ] Verify test configuration cannot target production databases.
- [ ] Document which requirements are covered at unit vs. integration levels.

Quick Mastery Check

1. A repository query must be verified against PostgreSQL. Which test is most appropriate?

- *Option A*: A unit test with a mocked repository.
- *Option B (Correct)*: An integration test using PostgreSQL (e.g. via Testcontainers).
- *Option C*: A test that only checks HTTP status.
- *Rationale*: A real database engine is required to validate SQL dialect, type mappings, and constraint enforcement.

2. A test passes alone but fails when executed in the full test suite. What should you investigate first?

- *Option A*: Increase timeouts.
- *Option B*: Delete the failing assertion.
- *Option C (Correct)*: Shared mutable static state, execution order dependence, or uncleaned database records.
- *Rationale*: Suite-only failures point to contamination or side effects leaking across test boundaries.

3. What is the primary benefit of a narrow Spring test slice like `@WebMvcTest`?

- *Option A*: It eliminates the need for assertions.
- *Option B (Correct)*: It instantiates only the presentation layer, keeping tests fast and focused.
- *Option C*: It proves every backend database query works.
- *Rationale*: Slices provide fast feedback on HTTP contracts without booting unneeded database beans.

---

15. JUnit 5 Completion Checklist

- [ ] Explain the JUnit Platform, Jupiter, and Vintage architecture.
- [ ] Write readable tests using the Arrange–Act–Assert pattern.
- [ ] Use assertions, parameterized tests, and lifecycle callbacks appropriately.
- [ ] Isolate service behavior with Mockito mocks and stubs.
- [ ] Choose the correct Spring testing scope (`@WebMvcTest`, `@DataJpaTest`, `@SpringBootTest`).
- [ ] Test persistence behavior using real database semantics (`flush()` and `clear()`).
- [ ] Diagnose flaky and environment-dependent CI failures.
- [ ] Explain the testing pyramid and its architectural trade-offs.
- [ ] Run and debug tests reliably through Maven (`./mvnw test`).
- [ ] Articulate testing trade-offs in backend engineering interviews.

Final Completion Protocol

1. Authoritative source files:
- `content/junit5/01-junit5-fundamentals-unit-testing.md`
- `content/junit5/02-junit5-advanced-testing-production.md`
2. Both chunks verified for all three learning layers.
3. Single final handbook PDF: `docs/junit5/BEH_JUNIT5_COMPLETE_HANDBOOK.pdf`.
4. Update `README.md` progress tracker.
5. Commit with exact message: `JUnit 5 done`.